

Spring MVC

WebApp 2 → 4 Study Notes

Class-level Routing Context Path JSTL POJO Models

Contents

Section	Topic	WebApp
1	Class-level @RequestMapping & Multiple Controllers	2
2	Same URL, Different HTTP Methods	2
3	Context Path (server.servlet.context-path)	3
4	Passing Arrays to the View	3
5	JSTL c:forEach — Looping in JSP	3
6	Multiple JSP Views & Navigation	3
7	POJO / Model Class (JavaBean Pattern)	4
8	Passing Objects vs Individual Values	4
9	EL Dot Notation & Getters	4
10	Complete MVC Layer Separation	4

Class-Level Routing & Multiple Controllers

Namespacing URLs with `@RequestMapping` on the class

1 Class-Level `@RequestMapping`

Previously `@RequestMapping` was only placed on individual methods. In WebApp 2 it is placed on the **class itself**, acting as a URL prefix for every method inside that controller.

```
@Controller

@RequestMapping("first")           // class-level prefix

public class FirstController {

    @RequestMapping("/telusko1")    // full URL: /first/telusko1

    public ModelAndView displayInfo() { ... }

    @PostMapping("/course-info")    // full URL: /first/course-info (POST)

    public ModelAndView displayInfo2() { ... }

    @GetMapping("/course-info")     // full URL: /first/course-info (GET)

    public ModelAndView displayInfo24() { ... }

}
```

URL resolution formula:

URL = class prefix + method path

"first" + "/telusko1" = /first/telusko1

"first" + "/course-info" = /first/course-info

```
"second" + "/course-info" = /second/course-info
```

2 Multiple Controllers

Spring MVC supports any number of controllers. Each controller handles a different area of the application. The class-level prefix keeps their URLs unique even when method paths overlap.

```
@Controller

@RequestMapping("second")          // different prefix

public class SecondController {

    @GetMapping("/course-info")    // full URL: /second/course-info

    public String getCourseInfo(Model model) {

        model.addAttribute("message", "DevOps with AWS course launched");

        return "index";

    }

}
```

Controller	Class Prefix	Method	Full URL
FirstController	/first	@GetMapping("/course-info")	/first/course-info
FirstController	/first	@PostMapping("/course-info")	/first/course-info (POST)
SecondController	/second	@GetMapping("/course-info")	/second/course-info

■ ■ Without the class-level prefix, having two controllers with the same `@GetMapping("/course-info")` would throw an `AmbiguousHandlerMappingException` at startup.

3 Same URL, Different HTTP Methods

A single URL can be mapped to **multiple methods** as long as each method handles a different HTTP verb. Spring routes the request to the correct method based on the verb.

```
// Both map to /first/course-info but for different verbs
```

```

@PostMapping("/course-info")

public ModelAndView handlePost() {

    mav.addObject("message", "POST: visit learn.telusko.com");

    mav.setViewName("index");

    return mav;

}

@GetMapping("/course-info")

public ModelAndView handleGet() {

    mav.addObject("message", "GET: visit learn.telusko.com");

    mav.setViewName("index");

    return mav;

}

```

Scenario	HTTP Status	Reason
URL matches, verb matches	200 OK	Correct method is invoked
URL matches, verb doesn't match	405 Method Not Allowed	No method for that verb
URL doesn't match at all	404 Not Found	No handler found

■ ■ This is standard REST practice: GET /course-info loads the page, POST /course-info submits a form to the same URL.

WebApp 3

Context Path, Arrays & JSTL

Global URL prefix, passing arrays to views, looping with JSTL

4 Context Path

The **context path** is a global prefix applied to every URL in the application. It is set once in **application.properties** and affects all endpoints.

```
# application.properties

server.port=8484

server.servlet.context-path=/telusko    # global prefix

spring.mvc.view.prefix=/view/

spring.mvc.view.suffix=.jsp
```

Effect on URLs:

Without context-path vs With context-path

http://localhost:8484/home

http://localhost:8484/telusko/home

http://localhost:8484/telusko/response

Setting	Scope	Where Configured	Example
context-path=/telusko	Entire application	application.properties	/telusko/home
@RequestMapping("first")	One controller	Class annotation	/first/course-info
@GetMapping("/home")	One endpoint	Method annotation	/home

■ ■ Use context-path to deploy multiple apps on the same server without URL collisions. e.g. /telusko, /myapp, /admin all on port 8080.

5 Passing Arrays to the View

The model can hold **any Java type** — not just Strings. Here an array of Strings is added to the model and iterated in the JSP.

```
@GetMapping("/response")

public String showCourses(Model model) {

    String books[] = new String[]{"Java","DevOps","AWS","AI Engineering"};

    model.addAttribute("books", books); // key="books", value=array

    return "response";

}
```

In the JSP, **`${books}`** refers to the full array. You use JSTL to iterate it:

6

JSTL `c:forEach` — Looping in JSP

JSTL (JSP Standard Tag Library) provides HTML-like tags for logic in JSP pages. It replaces raw Java code inside JSP, keeping the view layer clean.

Step 1 — Import the JSTL taglib at the top of the JSP

```
<%@taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c"%>
```

Step 2 — Use `c:forEach` to iterate

```
<c:forEach var="booksName" items="${books}">

    <div class="book-item">${booksName}</div>

</c:forEach>
```

Attribute	Value	Meaning
items	<code>\${books}</code>	The array/list from the model
var	<code>booksName</code>	Variable name for each element in the current iteration
(body)	<code>\${booksName}</code>	Renders the current element using Expression Language

Equivalent Java code (never do this in JSP!)

```
// Java equivalent – DO NOT put this in JSP

for(String booksName : books) {

    System.out.println(booksName);

}
```

■■ Never write Java loops or logic inside JSP using scriptlets (<% %>). Always use JSTL tags. Keep Java in Java, keep rendering in JSP.

7 Multiple JSP Views & Navigation

WebApp 3 introduced two separate views — **home.jsp** and **response.jsp**. Each controller method returns a different view name.

```
@GetMapping("/home")

public String showHome() {

    return "home";        // resolves to /view/home.jsp

}

@GetMapping("/response")

public String showCourses(Model model) {

    // ...add model data...

    return "response";    // resolves to /view/response.jsp

}
```

Navigation between pages using a relative link in home.jsp:

```
<a href="response">Click here to get response</a>

<!-- Browser resolves this to: /telusko/response -->
```

■ ■ The link `href="response"` is a relative URL — it resolves relative to the current path `/telusko/home`, giving `/telusko/response`. This is how multi-page navigation works in Spring MVC.

POJO Models & Object Passing

JavaBean pattern, EL dot notation, complete MVC separation

8 POJO / Model Class (JavaBean Pattern)

WebApp 4 introduces a **dedicated model package** with a **Course** class. This is a **POJO (Plain Old Java Object)** — also called a **JavaBean**. It represents the 'M' in MVC.

```
package com.telusko.model;

public class Course {

    private Integer id;        // private fields – encapsulation

    private String  name;

    private Double  cost;

    // No-arg constructor – required by Spring / JPA

    public Course() {}

    // Parameterised constructor – for easy creation

    public Course(Integer id, String name, Double cost) {

        this.id = id; this.name = name; this.cost = cost;

    }

    // Public getters – MANDATORY for EL ${course.id}
```

```

    public Integer getId()    { return id; }

    public String  getName() { return name; }

    public Double  getCost() { return cost; }

    // Public setters – for data binding / forms

    public void setId(Integer id)    { this.id = id; }

    public void setName(String name) { this.name = name; }

    public void setCost(Double cost)  { this.cost = cost; }

}

```

JavaBean Rules (must follow all four)

- Fields are **private**
- Each field has a **public getter** named get<FieldName>()
- Each field has a **public setter** named set<FieldName>()
- Class has a **no-arg (empty) constructor**

9 Passing Objects vs Individual Values

WebApp 4 demonstrates both approaches side by side. Comparing them reveals why objects are always preferred in real applications.

Approach A — Individual values via Map (/course-info)

```

@GetMapping("/course-info")

public String getCourseInfo(Map<String, Object> map) {

    map.put("cid",    4);

    map.put("cname", "AI Engineering");
}

```

```
map.put("cost", 3499.4);

return "courseinfo";

}
```

JSP accesses each key directly:

```
<div>${cid}</div>

<div>${cname}</div>

<div>${cost}</div>
```

Approach B — Full object via Model (/course-details)

```
@GetMapping("/course-details")

public String getCourseInfo(Model model) {

    Course course = new Course(3, "DevOps", 2499.5);

    model.addAttribute("course", course); // ONE entry

    return "course";

}
```

JSP uses dot notation to access object properties:

```
<div>${course.id}</div>

<div>${course.name}</div>

<div>${course.cost}</div>
```

	Map / Individual	Object (POJO)
Model entries	One per field (3 for Course)	Just 1
JSP syntax	\${cid}, \${cname}	\${course.id}, \${course.name}

Scalability	Messy with many fields	Clean regardless of field count
Type safety	No — everything is Object	Yes — strongly typed
Real-world usage	Rarely	Almost always

■ ■ Always prefer passing objects. Imagine a Course with 15 fields — that is 15 `map.put()` calls vs one `model.addAttribute()`.

10 EL Dot Notation & Getters

Expression Language (EL) dot notation does **not** access fields directly. It calls the corresponding **getter method** behind the scenes.

EL Expression	What Spring Actually Calls	Returns
<code>\${course.id}</code>	<code>course.getId()</code>	3
<code>\${course.name}</code>	<code>course.getName()</code>	"DevOps"
<code>\${course.cost}</code>	<code>course.getCost()</code>	2499.5

■ ■ If you remove `getId()` from the Course class, `${course.id}` silently renders blank. No error is thrown — this is one of the most common bugs beginners encounter.

The EL Resolution Process

1. EL sees `${course.id}`
2. It looks up the attribute **"course"** in the model (request scope)
3. It calls **`course.getId()`** using reflection
4. The return value is rendered as a String in the HTML

11 Complete MVC Layer Separation

WebApp 4 is the first example with all three MVC layers clearly separated into distinct packages and files.

Layer	Package / File	Class / File	Responsibility
Model (M)	<code>com.telusko.model</code>	<code>Course.java</code>	Data structure, fields, getters/setters
Controller (C)	<code>com.telusko.controller</code>	<code>TeluskoController.java</code>	Request handling, business logic
View (V)	<code>/view/</code>	<code>course.jsp</code>	HTML rendering, display only

The complete flow for **GET /telusko/course-details**:

Browser: GET http://localhost:8484/telusko/course-details

|

v

DispatcherServlet (Front Controller – receives all requests)

|

v

HandlerMapping (finds @GetMapping("/course-details") in TeluskoController)

|

v

Controller (creates new Course(3,"DevOps",2499.5))

(model.addAttribute("course", course))

(returns "course")

|

v

ViewResolver (/view/ + course + .jsp = /view/course.jsp)

|

v

JSP View (\${course.id} -> getId() -> 3)

(\${course.name} -> getName() -> DevOps)

(\${course.cost} -> getCost() -> 2499.5)

|

v

Browser: receives final rendered HTML page

Quick Reference — WebApp 2 to 4

@RequestMapping on class	Acts as URL prefix for all methods in the controller
Multiple @Controllers	Each handles a different area; class prefix keeps URLs unique
Same URL + different verbs	@GetMapping and @PostMapping can share the same path
context-path	Global URL prefix for entire app, set in application.properties
model.addAttribute()	Adds any Java object (String, array, POJO) to the model
JSTL <%@taglib%>	Must import before using any tags in JSP
	Loops over arrays/lists from the model; use instead of Java scriptlets
POJO / JavaBean	Private fields + public getters/setters + no-arg constructor
EL \${course.name}	Calls course.getName() — getters are mandatory for EL to work
Pass objects not values	One model.addAttribute("course", course) > 10 map.put() calls
MVC Separation	Model (data) Controller (logic) View (rendering) — never mix